

Agent spend governance on Cloudflare

A small build on the agent-provisioning protocol, and what I learned making it · Nivedita Thapa

Your post on agents provisioning and paying on Cloudflare ended with an invitation to build something and show it. The part that stuck with me was the payment section, specifically the question you raised yourselves: *what if my agent goes a bit overboard and starts buying dozens of domains?* The answer today is a flat \$100/month cap. So I built the layer that sits where that cap is, to see what governing agent spend actually takes. Here is what I made and what I think.

What I built

A Cloudflare Worker that governs an agent's provisioning spend with explicit rules instead of one number. For each action an agent attempts (register a domain, add a bucket, start a subscription) it returns one of three verdicts, and logs the reason:

- **approve** — within budget and within an auto-approve rule; the agent proceeds without a human.
- **hold** — affordable, but routed to a human first (any recurring charge, or a one-time charge over a threshold).
- **deny** — would exceed the remaining budget.

The budget ledger and a full audit trail persist in Workers KV. A simulated agent drives a realistic purchase queue against it, and there is an approve path for a human to release held actions. The policy logic is isolated and tested, so the decisions are inspectable rather than buried in request handling.

I deliberately stubbed the Stripe identity attestation and payment-token exchange. The protocol already solves those well, and re-implementing OAuth and tokenization would have added plumbing without adding product insight. The interesting question lives entirely in the governance layer, so that is where I spent the time. The stub is marked explicitly in the code.

Why a flat cap isn't enough

Building it made the core problem concrete: a single dollar limit treats every charge as identical, and they aren't. Three things separate one charge from another, and a cap sees none of them.

- **Recurrence.** A one-time \$12 domain and a \$12/month subscription look the same to a cap on the day they happen. One is done; the other compounds into a bill nobody decided to keep paying. The surprise bills people actually get burned by are almost always recurring, so in my policy recurring charges always route to a human, regardless of size.
- **Reversibility.** Buying a domain is hard to undo; spinning up a bucket isn't. The more irreversible the action, the more a human should be in the loop, independent of price.
- **Headroom.** A \$15 charge with \$85 of budget left is routine; the same charge with \$8 left is the one that strands the agent mid-task. The decision should account for what is remaining, not just the sticker price.

The product feedback

Three things I'd raise, in priority order, from having built against the protocol:

- **The catalog should carry charge semantics, not just price.** To govern well, my layer needed to know whether a charge was one-time or recurring, and how reversible it was. Today that has to be inferred. If Discovery returned a little structure per service (recurring, a rough reversibility class), governance gets much better for free, and agents make better choices too.
- **Budget needs a notion of “ask me,” not just “stop.”** A cap is a wall: under it everything passes silently, over it everything fails. The more useful primitive is a hold — let the agent keep working while a specific charge waits for a one-tap human yes. That keeps the human in control without making them babysit, which feels closer to the “human in the loop to grant permission” framing in the post.
- **The audit trail is a product surface, not just a log.** Once agents spend autonomously, “what did my agent buy this month and why was each one allowed” becomes a question every customer asks. The decision-plus-reason record I built for governance turns out to be the thing a human most wants to see afterward. Worth treating as first-class, not a debugging byproduct.

How this connects to where Cloudflare is going

The same judgment that decides whether an agent should buy a domain is the judgment that decides whether an agent should pay to crawl a piece of content under Pay per Crawl. Both are an agent spending a constrained budget against a catalog of priced things, where some purchases should be automatic and some should not. Governing agents as economic actors looks like one problem from two directions, provisioning on one side and content on the other, and a clean answer on the developer-platform side is reusable on the AI Access side. That overlap is a large part of why this problem interested me.

Code, tests, a from-scratch deploy guide, and an offline runner are in the repo. The policy engine is the part I'd most want to talk through; the thresholds are deliberately simple, and I have opinions on where they'd need to get smarter for real customers.